

Projet_apprentissage

May 31, 2025

1 Reinforcement Learning Project : N7Craft

1.1 Training a Minecraft Jump Agent with Mineflayer & RL

A Gym-style Env, PPO Training Loop, and Evaluation on Custom Jump Maps

1.1.1 Team Members

- Corentin Cousty
 - Hermas Obou
 - Ignacio Arroyo
 - Wilkens Joseph
-

Project Goal:

This notebook demonstrates how to train a reinforcement learning (RL) agent to complete custom Minecraft “jump” courses. A “jump” in Minecraft is a parkour-like sequence requiring precise movements and timing to land on successive blocks, with the possibility to add other mechanics like ladders or specific game features.

High-level flow:

- Connect a Mineflayer bot to a custom Minecraft server with a tailored jump map. - Wrap the environment in a Gym-compatible API to facilitate RL experimentation. - Train an agent using Proximal Policy Optimization (PPO), a popular RL algorithm. - Evaluate the agent’s performance and ability to generalize to new jump maps.

Team context:

This project was realized by Corentin Cousty, Hermas Obou, Ignacio Arroyo, and Wilkens Joseph as part of the “Contrôle et Apprentissage” course at ENSEEIHT-INP Toulouse. The goal is for the agent to learn—through trial and error—how to reach the goal block, receiving rewards based on its progression, using only information about blocks and positions (no image recognition needed).

1.2 Environment Setup & Imports

Launch the Minecraft Server (version 1.19.4 if you want to connect to it as a player). It’s going to open a java console for managing the server.

```
[1]: import subprocess

# Start in background, no console output
subprocess.Popen(
    ['java', '-Xms1G', '-Xmx2G', '-jar', 'paper-1.19.4-550.jar'],
    cwd='minecraft-server',
    stdout=subprocess.DEVNULL,
    stderr=subprocess.STDOUT
)
```

[1]: <Popen: returncode: None args: ['java', '-Xms1G', '-Xmx2G', '-jar', 'paper-1...>

Install all required libraries:

```
[2]: # Use `n` to install nodejs 18, if it's not already installed:
#!curl -fsSL https://raw.githubusercontent.com/tj/n/master/bin/n | bash -s lts
    ↪> /dev/null
# Now write the Node.js and Python version to the console
!node --version
!python --version
```

v22.13.1

Python 3.13.1

```
[3]: !pip install javascript
```

Requirement already satisfied: javascript in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages
(1!1.2.2)

```
[4]: !pip install gym vec3 javascript stable-baselines3 torch matplotlib numpy
```

Requirement already satisfied: gym in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages
(0.26.2)
Requirement already satisfied: vec3 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (0.1.0)
Requirement already satisfied: javascript in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages
(1!1.2.2)
Requirement already satisfied: stable-baselines3 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (2.6.0)
Requirement already satisfied: torch in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (2.7.0)
Requirement already satisfied: matplotlib in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages
(3.10.0)
Requirement already satisfied: numpy in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (2.2.2)

Requirement already satisfied: cloudpickle>=1.2.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
gym) (3.1.1)

Requirement already satisfied: gym_notices>=0.0.4 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
gym) (0.0.8)

Requirement already satisfied: gymnasium<1.2.0,>=0.29.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
stable-baselines3) (1.1.1)

Requirement already satisfied: pandas in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
stable-baselines3) (2.2.3)

Requirement already satisfied: filelock in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (3.17.0)

Requirement already satisfied: typing-extensions>=4.10.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (4.12.2)

Requirement already satisfied: sympy>=1.13.3 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (1.13.3)

Requirement already satisfied: networkx in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (3.4.2)

Requirement already satisfied: jinja2 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (3.1.6)

Requirement already satisfied: fsspec in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (2025.2.0)

Requirement already satisfied: setuptools in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (75.8.0)

Requirement already satisfied: farama-notifications>=0.0.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
gymnasium<1.2.0,>=0.29.1->stable-baselines3) (0.0.4)

Requirement already satisfied: contourpy>=1.0.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (1.3.1)

Requirement already satisfied: cycler>=0.10 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (4.56.0)

Requirement already satisfied: kiwisolver>=1.3.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (1.4.8)

Requirement already satisfied: packaging>=20.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (24.2)

Requirement already satisfied: pillow>=8 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (11.1.0)

Requirement already satisfied: pyparsing>=2.3.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (3.2.1)

Requirement already satisfied: python-dateutil>=2.7 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
python-dateutil>=2.7->matplotlib) (1.17.0)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
jinja2->torch) (3.0.2)

Requirement already satisfied: pytz>=2020.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
pandas->stable-baselines3) (2024.2)

Requirement already satisfied: tzdata>=2022.7 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
pandas->stable-baselines3) (2025.1)

1.3 Visualization Helpers

Provide functions to help to debug and visualize:

- Plotting the block-ID matrix as a heatmap
- Rendering a simple textual or graphical view of the agent's surrounding - Used for the render function of the env.

```
[5]: import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # noqa: unused
import numpy as np

def plot_block_volume(vol):
    """
    vol: a (X×Y×Z) np.ndarray of block IDs around the bot.
    We'll draw a voxel wherever block ID > 0.
    """
    filled = vol > 0

    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
```

```

ax.voxels(filled)                                # default facecolors
ax.set_xlabel('X (left-right)')
ax.set_ylabel('Y (down-up)')
ax.set_zlabel('Z (behind-front)')
ax.set_title('3D Block Volume')
plt.show()

def render_agent_view_3d(env):
    """Grab your 3D window and plot it."""
    vol = env._get_volume()                        # your existing sampler
    plot_block_volume(vol)

```

1.4 Environment Definition

This section defines all the key parameters and constants that configure the Minecraft jump environment for RL training:

- **Special Block Types** (used for navigation and rewards):
 - `START_BLOCK_NAME`: The block marking the agent’s spawn/start (e.g., `iron_block`).
 - `PATH_BLOCK_NAME`: The block type forming the walkable path/platforms (e.g., `stone`).
 - `CHECKPOINT_BLOCK_NAME`: Used for intermediate reward points (e.g., `gold_block`).
 - `FINISH_BLOCK_NAME`: Block that marks the end/goal of the jump course (e.g., `diamond_block`).
 - `DEATH_BLOCK_NAME`: Deadly/failure block (e.g., `obsidian`).
- **Reward Parameters:**
 - `PLATFORM_BONUS`: Reward for discovering a new platform.
 - `FINISH_REWARD`: Large reward for reaching the finish block.
 - `DEATH_PENALTY`: Negative reward for dying or falling.
 - `STEP_PENALTY`: Small penalty per time step to encourage faster solutions.
 - `CLOSER_TO_PATH_BONUS`: Shaping reward for moving closer to the path/goal.
 - `NOT_LOOKING_TOWARD_PLATFORM_PENALTY`: Penalty if the agent is not facing towards the next platform.
- **Environment Geometry:**
 - `MAX_PATH_RADIUS`: Maximum search radius (in blocks) for pathfinding/reward shaping.
 - `LOOKING_TOWARD_PLATFORM_THRESHOLD`: Cosine threshold for determining if the agent is looking towards a platform.
 - `MAX_STEPS_ON_PLAT`: Max steps allowed on a platform before being penalized for stalling.
- **Jump Courses:**
 - Each jump map is defined by its start and finish coordinates (see the `JUMPS` list for examples).

These parameters collectively define the dynamics, rewards, and spatial constraints of the RL environment for the Mineflayer agent.

```

[6]: # === Minecraft Jump RL Environment Core (Reward, Done, Teleport/Respawn) ===
import math
import os

```

```

HOST, PORT = "localhost", 25565

# Names for special blocks from infos.txt
START_BLOCK_NAME      = "iron_block"      # Iron block (start)
PATH_BLOCK_NAME        = "stone"          # Stone (path)
CHECKPOINT_BLOCK_NAME  = "gold_block"      # Gold block (checkpoint)
FINISH_BLOCK_NAME      = "diamond_block"   # Diamond block (finish)
DEATH_BLOCK_NAME       = "obsidian"        # Obsidian (death)
MAX_STEPS_ON_PLAT      = 4

# Rewards parameters:
PLATFORM_BONUS         = 30
FINISH_REWARD          = 500
DEATH_PENALTY          = -50
STEP_PENALTY           = -1
MAX_PATH_RADIUS        = 10    # how far (in blocks) to search
CLOSER_TO_PATH_BONUS   = 3
NOT_LOOKING_TOWARD_PLATFORM_PENALTY = -2    # penalty for not facing towards the
↳nearest platform
LOOKING_TOWARD_PLATFORM_THRESHOLD = math.cos(math.radians(45))    # require
↳dot(facing, to_platform)    this (cos(45°))

Jump1 = {
    "start": (8, -55, 12),
    "finish": (8, -55, -12),
}
Jump2 = {
    "start": (-13, -55, 12),
    "finish": (-13, -54, -12),
}
Jump3 = {
    "start": (-29, -55, 9),
    "finish": (-29, -54, -16),
}
Jump4 = {
    "start": (-36, -55, 54),
    "finish": (-15, -52, 30),
    "checkpoint1" : (-36, -54, 30),
}

JUMPS = [Jump1, Jump2, Jump3, Jump4]

```

1.5 Gym-Style Environment Class

The core of our RL framework is a custom OpenAI Gym environment wrapping the Mineflayer bot. This class exposes the Minecraft jump scenario as a standard RL environment:

- **Observation Space:**
 - "vision3d": A $(9 \times 5 \times 9)$ matrix (tensor) encoding block IDs in a 3D region around the agent (± 4 left/right, ± 2 up/down, 3 behind, 6 ahead).
 - "position": The bot's (x, y, z) float coordinates.
- **Action Space:**
 - MultiBinary(n): Each action is represented as a binary vector with a length depending on enabled rotations. By default:
 - * [forward, jump, sprint, turn_small_left, turn_small_right, ...]
 - * Supports combinations (move forward & jump, etc.)
- **Implemented Methods:**
 - `__init__` — Initializes the environment, connects the bot, sets up observation/action spaces, resets state.
 - `reset` — Teleports the agent to the start, resets exploration and counters, returns initial observation.
 - `step` — Applies action vector to bot (movement & rotation), steps simulation, computes reward and done, returns new observation and info.
 - `_compute_reward_done` — Calculates reward and checks for termination (fall, death, finish, etc.).
 - `_get_volume` — Samples the block IDs in the agent's field of view for observation.
 - `_nearest_path_coord` — Finds nearest undiscovered path block (for shaping rewards/navigation).
 - `_maybe_discover_platform` — Flood-fills platforms to handle reward bonuses for first visits.
 - `_angle_penalty` — Computes penalties for not looking towards the next platform.
 - `render` — Visualizes the agent's local environment.
 - `close` — Cleans up, disconnects the bot from the server.

This modular design allow to easily train RL agents using Stable Baselines3 (PPO, DQN, etc.) on complex Minecraft jump courses, while leveraging custom reward shaping and 3D observations.

```
[7]: import gymnasium as gym
from gymnasium import spaces

from collections import deque
from itertools import product

def agent_on_block_name(bot, block_name):
    pos = bot.entity.position
    y = math.floor(pos.y - 0.7)
    for dx in [-0.3, 0.3]:
        for dz in [-0.3, 0.3]:
            x = math.floor(pos.x + dx)
```

```

        z = math.floor(pos.z + dz)
        block = bot.blockAt(Vec3(x, y, z))
        if block and block.name == block_name:
            return True
    return False

class MinecraftRL(gym.Env):
    """
    Dynamic action space:
    - 0: forward
    - 1: backward
    - 2: jump
    - 3: sprint
    - then for each rotation in rotation_options:
        turn_<opt>_left, turn_<opt>_right
    Observation includes:
    - vision3d: a (9×5×9) block-ID tensor around the bot
    - position: float (x,y,z)
    """
    metadata = {'render.modes': ['human']}

    def __init__(self, bot, jump_id=0,
                 turn_delta=math.pi/8,
                 rotation_options=('small', 'medium', 'big'), frame_skip=4):
        super().__init__()
        self.frame_skip = frame_skip

        self.bot = bot
        self.jump = JUMPS[jump_id]
        self.spawn = self.jump["start"]

        self.teleport_to_start()

        # reward & block definitions
        self.death_block_name = DEATH_BLOCK_NAME
        self.finish_block_name = FINISH_BLOCK_NAME
        self.PLATFORM_BONUS = PLATFORM_BONUS
        self.FINISH_REWARD = FINISH_REWARD
        self.DEATH_PENALTY = DEATH_PENALTY
        self.STEP_PENALTY = STEP_PENALTY
        self.MAX_STEPS_ON_PLAT = MAX_STEPS_ON_PLAT
        self.CLOSER_TO_PATH_BONUS = CLOSER_TO_PATH_BONUS
        self.MAX_PATH_RADIUS = MAX_PATH_RADIUS
        self.NOT_LOOKING_TOWARD_PLATFORM_PENALTY = _
        ↪NOT_LOOKING_TOWARD_PLATFORM_PENALTY
        self.LOOKING_TOWARD_PLATFORM_THRESHOLD = _
        ↪LOOKING_TOWARD_PLATFORM_THRESHOLD

```



```

# define available angle magnitudes
all_angles = {
    'small': turn_delta,
    'medium': math.pi/4,
    'big': math.pi/2,
}
# filter to only those the user asked for
self.rotation_options = [opt for opt in rotation_options if opt in
↪all_angles]
self.angles = {opt: all_angles[opt] for opt in self.rotation_options}

# build action names
self.ACTION_NAMES = ['forward', 'back', 'jump', 'sprint']
for opt in self.rotation_options:
    self.ACTION_NAMES += [f'turn_{opt}_left', f'turn_{opt}_right']

# spaces
self.action_space = spaces.MultiBinary(len(self.ACTION_NAMES))

# 3D vision spans
self.span_x = 4 # ±4 left/right → width = 9
self.span_y = 2 # ±2 up/down → height=5
self.behind = 3 # 3 behind
self.front = 6 # 6 in front → depth = 9

vision_shape = (
    2*self.span_x + 1,
    2*self.span_y + 1,
    self.behind + self.front
)

self.observation_space = spaces.Dict({
    "vision3d": spaces.Box(0, 4096,
                           shape=vision_shape,
                           dtype=np.int32),
    "position": spaces.Box(low = np.array([-1000, 0, -1000], dtype=np.
↪float32),
                           high= np.array([ +1000, 256, +1000],
↪dtype=np.float32),
                           shape=(3,),
                           dtype=np.float32),
})

# runtime state
self.discovered = set()
self.steps_on_current_platform = 0

```

```

self.last_path_dist = self.MAX_PATH_RADIUS
self.fall_y_limit = self.spawn[1] - 2

# teleport & effects & face
x,y,z = self.spawn
name = self.bot.username

self.bot.chat(f"/effect give {name} resistance 2147483647 3 true")
self.bot.chat(f"/effect give {name} saturation 2147483647 3 true")
self.bot.chat(f"/effect give {name} regeneration 2147483647 3 true")
self._face_nearest_platform()

self.bot.chat("/spawnpoint")

def _face_nearest_platform(self):
    # 1) find nearest *new* path coord in 3D
    tgt = self._nearest_path_coord()
    if tgt is None:
        return

    # 2) compute yaw toward that block
    px = self.bot.entity.position.x
    pz = self.bot.entity.position.z
    tx, _, tz = tgt
    dx, dz = tx - px, tz - pz
    yaw_rad = math.atan2(-dx, dz)

    # 3) convert to Minecraft yaw in degrees [0,360)
    yaw_deg = (math.degrees(yaw_rad) + 360) % 360

    # 4) teleport in place, set pitch to 0
    self.bot.chat(f"/tp {self.bot.username} ~ ~ ~ {yaw_deg:.1f} 0")

def reset(self, **kwargs):
    super().reset(**kwargs)
    self.teleport_to_start()
    name = self.bot.username
    self.bot.chat(f"/effect give {name} resistance 2147483647 3 true")
    self.bot.chat(f"/effect give {name} saturation 2147483647 1 true")
    self._face_nearest_platform()
    self.discovered.clear()
    self.steps_on_current_platform = 0
    self.last_path_dist = self.MAX_PATH_RADIUS
    return self.get_observation(), {}

def step(self, action):

```

```

total_reward = 0
done = False
info = {}
obs = None

for _ in range(self.frame_skip):
    self.steps_on_current_platform += 1

    self.bot.setControlState('forward', bool(action[0]))
    self.bot.setControlState('back', bool(action[1]))
    if action[2]:
        self.bot.setControlState('jump', True)
        self.bot.setControlState('jump', False)

    self.bot.setControlState('sprint', bool(action[3]))

    # rotation
    yaw = self.bot.entity.yaw
    base = 4
    for i, opt in enumerate(self.rotation_options):
        left_idx = base + 2*i
        right_idx = base + 2*i + 1
        if action[left_idx]: yaw -= self.angles[opt]
        if action[right_idx]: yaw += self.angles[opt]
    self.bot.look(yaw, self.bot.entity.pitch)

    obs, reward, done, _, info = self._after_motion()
    total_reward += reward

    if done:
        break

return obs, total_reward, done, False, info

def _get_foot_coord(self):
    """Return the central block under the agent's feet as an (x,y,z) triple.
    ↪ """
    pos = self.bot.entity.position
    # floor to the block grid:
    x = math.floor(pos.x)
    y = math.floor(pos.y - 0.7)
    z = math.floor(pos.z)
    return (x, y, z)

def _bfs_platform(self, start):
    """Flood-fill the stone platform starting at `start`; return set of
    ↪ coords."""

```

```

frontier = deque([start])
plat = {start}
while frontier:
    x,y,z = frontier.popleft()
    for dx,dz in [(1,0),(-1,0),(0,1),(0,-1)]:
        nb = (x+dx, y, z+dz)
        if nb not in plat:
            blk = self.bot.blockAt(Vec3(*nb))
            if blk and blk.name == PATH_BLOCK_NAME:
                plat.add(nb)
                frontier.append(nb)

return plat

def teleport_to_start(self):
    x, y, z = self.jump["start"]
    success = False
    attempts = 0

    name = self.bot.username

    while (self.bot.entity.position.y < y - 1):
        self.bot.chat(f"/tp {name} {int(x)} {int(y)} {int(z)}")
        success = True

def _get_volume(self):
    """Builds a (X*Y*Z) numpy array of block.type around the bot."""
    X = 2*self.span_x + 1
    Y = 2*self.span_y + 1
    Z = self.behind + self.front
    vol = np.zeros((X, Y, Z), dtype=np.int32)

    pos = self.bot.entity.position
    px, py, pz = pos.x, pos.y, pos.z
    yaw = self.bot.entity.yaw

    # compute local axes as simple tuples
    forward = (-math.sin(yaw), 0.0, -math.cos(yaw))
    right = (-forward[2], 0.0, forward[0])
    up = (0.0, 1.0, 0.0)

    for ix, dx in enumerate(range(-self.span_x, self.span_x+1)):
        for iy, dy in enumerate(range(-self.span_y, self.span_y+1)):
            for iz, dz in enumerate(range(-self.behind, self.front)):
                # Python-only arithmetic
                ox = right[0]*dx + up[0]*dy + forward[0]*dz
                oy = right[1]*dx + up[1]*dy + forward[1]*dz
                oz = right[2]*dx + up[2]*dy + forward[2]*dz

```

```

        # Now build a Vec3 only for blockAt
        blkpos = Vec3(px + ox, py + oy, pz + oz)
        blk = self.bot.blockAt(blkpos)
        vol[ix, iy, iz] = blk.type if blk else 0

    return vol

def get_observation(self):
    vis3d = self._get_volume()
    p      = self.bot.entity.position
    pos    = np.array([p.x, p.y, p.z], dtype=np.float32)
    return {"vision3d": vis3d, "position": pos}

def _nearest_path_coord(self):
    """
    3D BFS up to MAX_PATH_RADIUS to find the nearest *new* PATH_BLOCK_NAME,
    i.e. a block not yet in self.discovered. Returns (x,y,z) or None.
    """
    start = self._get_foot_coord()
    q      = deque([(start, 0)])
    seen   = {start}
    while q:
        (x, y, z), dist = q.popleft()
        if dist > self.MAX_PATH_RADIUS:
            break

        blk = self.bot.blockAt(Vec3(x, y, z))
        if blk and blk.name == PATH_BLOCK_NAME and (x, y, z) not in self.
↪discovered:
            return (x, y, z)

        # 6-neighborhood in x,y,z
        for dx, dy, dz in ↵
↪[(1,0,0),(-1,0,0),(0,1,0),(0,-1,0),(0,0,1),(0,0,-1)]:
            nb = (x+dx, y+dy, z+dz)
            if nb not in seen:
                seen.add(nb)
                q.append((nb, dist+1))

    return None

def _maybe_discover_platform(self, foot):
    """
    If `foot` is on a PATH_BLOCK and not yet discovered,
    flood-fill its contiguous platform and add to discovered.

```

```

        """
        if foot in self.discovered:
            return False

        blk = self.bot.blockAt(Vec3(*foot))
        if not (blk and blk.name == PATH_BLOCK_NAME):
            return False

        # flood-fill this entire platform (2D)
        plat = self._bfs_platform(foot)
        # merge into discovered
        self.discovered |= plat
        return True

def _angle_penalty(self, reward):
    # 1) find nearest *new* path block
    tgt = self._nearest_path_coord()
    if tgt is not None:
        tx, _, tz = tgt
    else:
        # fallback to finish if no path block nearby
        tx, _, tz = self.jump["finish"]

    # 2) compute horizontal vector to that target
    pos = self.bot.entity.position
    dx, dz = tx - pos.x, tz - pos.z
    dist = math.hypot(dx, dz)
    if dist < 1e-3:
        return reward
    dx, dz = dx/dist, dz/dist

    # 3) forward unit-vector from current yaw
    yaw = self.bot.entity.yaw
    fxz, fzz = -math.sin(yaw), math.cos(yaw)

    # 4) cosine of angle between them
    cos_theta = fxz*dx + fzz*dz

    # 5) apply penalty if outside acceptable look cone
    if cos_theta < self.LOOKING_TOWARD_PLATFORM_THRESHOLD:
        reward += self.NOT_LOOKING_TOWARD_PLATFORM_PENALTY

    return reward

def _distance_to_nearest_path(self):
    """

```

```

3D BFS up to MAX_PATH_RADIUS (Manhattan) to find distance to nearest
PATH_BLOCK_NAME (regardless of discovered-status).
"""
start = self._get_foot_coord()
q = deque([(start, 0)])
seen = {start}
while q:
    (x, y, z), dist = q.popleft()
    if dist > self.MAX_PATH_RADIUS:
        break

    blk = self.bot.blockAt(Vec3(x, y, z))
    if blk and blk.name == PATH_BLOCK_NAME:
        return dist

    for dx, dy, dz in ↪ [(1,0,0), (-1,0,0), (0,1,0), (0,-1,0), (0,0,1), (0,0,-1)]:
        nb = (x+dx, y+dy, z+dz)
        if nb not in seen:
            seen.add(nb)
            q.append((nb, dist+1))

return self.MAX_PATH_RADIUS

def _compute_reward_done(self, obs):
    # 0) Check for fall with y axis
    pos = self.bot.entity.position
    if pos.y < self.fall_y_limit:
        return self.DEATH_PENALTY, True

    # 1) death / finish
    if agent_on_block_name(self.bot, self.death_block_name):
        return self.DEATH_PENALTY, True
    if agent_on_block_name(self.bot, self.finish_block_name):
        return self.FINISH_REWARD, True

    # 2) baseline step penalty
    reward = self.STEP_PENALTY
    done = False

    # 3) platform bonus + reset TTL
    foot = self._get_foot_coord()
    if self._maybe_discover_platform(foot):
        reward += self.PLATFORM_BONUS
        self.steps_on_current_platform = 0

    # 4) TTL failure

```

```

    if self.steps_on_current_platform > self.MAX_STEPS_ON_PLAT:
        return self.DEATH_PENALTY, True

    # 5) distance-to-path shaping
    current_dist = self._distance_to_nearest_path()
    delta        = self.last_path_dist - current_dist
    reward       += delta * self.CLOSER_TO_PATH_BONUS
    self.last_path_dist = current_dist

    # 6) **view penalty** only if nearest target is a PATH_BLOCK_NAME
    reward = self._angle_penalty(reward)

    return reward, done

def _after_motion(self):
    """
    Called by step() once movement+look have been applied.
    Returns exactly the 5-tuple obs, reward, done, truncated, info
    that Gym expects.
    """
    # 1) get your observation
    obs = self.get_observation()
    # 2) compute reward and terminal-flag
    reward, done = self._compute_reward_done(obs)
    # 3) no truncation in this env, and empty info
    return obs, reward, done, False, {}

def render(self, mode='human'):
    vol = self._get_volume()
    plot_block_volume(vol)

def close(self):
    self.bot.quit()

```

1.6 Bot Connection Helper

Define a utility function to spawn and login the Mineflayer bot:

- Explain that this handles network connection and event waiting

We import Mineflayer & Vec3 (Node.js libs exposed via javascript package)

We define a function to spawn and login the Agent to the server in localhost.

```

[8]: import time
import socket
from javascript import require, once
import os, time, math, socket
from javascript import require

```



```

from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv, VecMonitor

Vec3 = require('vec3').Vec3

```

[JSE] (node:23152) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a userland alternative instead.

[JSE] (Use `node --trace-deprecation ...` to show where the warning was created)

```

[9]: # Helpers
def wait_for_server(host, port, timeout=60):
    t0 = time.time()
    while time.time() - t0 < timeout:
        try:
            s = socket.create_connection((host, port), timeout=2)
            s.close()
            return
        except OSError:
            time.sleep(1)
    raise TimeoutError(f"Server never opened port {port}")

def create_bot(host, port, username, login_timeout=10):
    """Blocks until this bot has actually logged in."""
    mineflayer = require('mineflayer')
    print("try to create a bot")
    bot = mineflayer.createBot({
        'host': host,
        'port': port,
        'username': username,
    })
    # poll on bot.entity as a proxy for "we're fully logged in"
    t0 = time.time()
    while time.time() - t0 < login_timeout:
        if hasattr(bot, 'entity') and bot.entity is not None:
            print(f" Logged in as {username}")
            return bot
        print("waiting to login")
        time.sleep(1)
    raise TimeoutError(f"Bot {username} never logged in")

def wait_for_bot_connection(host, port, username, retry_interval=1.0,
    ↪timeout=30.0):
    start = time.time()
    while True:
        try:

```

```

        return create_bot(host, port, username)
    except Exception as e:
        if time.time() - start > timeout:
            raise TimeoutError(f"Could not connect bot after {timeout}s_
↪(last error: {e!r})")
        print(f"Failed to connect bot ({e}); retrying in_
↪{retry_interval}s...")
        time.sleep(retry_interval)

```

1.7 Pre-spawn all the bots

```

[10]: LOG_DIR = "./RL_logs/"
NUM_ENVS = 2
def spawn_bots(n, host, port, timeout=120):
    bots = []
    for i in range(n):
        uname = f"RLBot_{i}"
        print(f"→ Spawning {uname}...")
        bot = wait_for_bot_connection(
            host=host,
            port=port,
            username=uname,
            timeout=timeout,
            retry_interval=2.0
        )
        print(f" {uname} connected!")
        bots.append(bot)
        bot.chat("/op " + "RLBot_" + str(i+1))
        time.sleep(1)
    return bots

wait_for_server(HOST, PORT)
# Usage:
bots = spawn_bots(NUM_ENVS, HOST, PORT)

```

```

→ Spawning RLBot_0...
try to create a bot
waiting to login
waiting to login
waiting to login
waiting to login
waiting to login
waiting to login
waiting to login
    Logged in as RLBot_0
    RLBot_0 connected!
→ Spawning RLBot_1...

```

```

try to create a bot
waiting to login
waiting to login
waiting to login
waiting to login
waiting to login
waiting to login
waiting to login
waiting to login
waiting to login
waiting to login
Failed to connect bot (Bot RLBot_1 never logged in); retrying in 2.0s...
try to create a bot
waiting to login
  Logged in as RLBot_1
  RLBot_1 connected!

```

1.8 Wrapping with Stable-Baselines3

To train our RL agent, we use the [Stable-Baselines3](#) library, which provides high-quality implementations of algorithms like PPO and DQN.

- **Instantiate the RL agent:**
 - We use the PPO (Proximal Policy Optimization) algorithm, which works well for environments with discrete or multi-binary actions and continuous state spaces.
 - The custom `MinecraftRL` environment is wrapped in a vectorized environment (`DummyVecEnv`), making it compatible with Stable-Baselines3 and allowing easy monitoring/logging.
 - Key hyperparameters (like `n_steps`, `batch_size`, `learning_rate`, and `gamma`) are chosen based on best practices for PPO and the environment’s complexity.

```

[11]: import threading

jump_train_id = 0
n_steps = 2048
print_freq = n_steps + 1
total_steps = n_steps * 40
save_freq = n_steps

def make_env(idx):
    def _init():
        bot = bots[idx]
        return MinecraftRL(
            bot,
            jump_id=jump_train_id,
            turn_delta=math.pi/8,
            rotation_options=(),
        )

```

```

    return _init

env_fns = [make_env(i) for i in range(NUM_ENVS)]
vec_env = DummyVecEnv(env_fns)
vec_env = VecMonitor(vec_env, LOG_DIR)

model = PPO(
    "MultiInputPolicy",
    vec_env,
    verbose = 1,
    n_steps = n_steps,
    batch_size = n_steps/4,
    learning_rate = 3e-4,
    gamma = 1,
)

```

Using cpu device

1.9 Training Loop

Training is performed using the `model.learn()` function from Stable-Baselines3:

- **Total Timesteps:**
 - Specify the total number of environment steps (e.g., 200_000) for the agent to interact and learn from the environment.
- **Logging and Checkpoints:**
 - Use callbacks (e.g., `CheckpointCallback`) to save the model at regular intervals (e.g., every 10,000 steps).
 - The vectorized environment is wrapped with `VecMonitor` to automatically record episode rewards, lengths, and additional metrics for analysis.

```

[12]: from stable_baselines3.common.callbacks import BaseCallback
      from stable_baselines3.common.callbacks import CheckpointCallback

class ProgressBarCallback(BaseCallback):
    def __init__(self, total_timesteps, print_freq=print_freq, verbose=0):
        super().__init__(verbose)
        self.total_timesteps = total_timesteps
        self.print_freq = print_freq

    def _on_step(self) -> bool:
        if self.n_calls % self.print_freq == 0 or self.n_calls == self.
↪total_timesteps:
            percent = 100 * self.n_calls / self.total_timesteps
            print(f"Training progress: {percent:.1f}% ({self.n_calls} / {self.
↪total_timesteps} steps)")
            return True

```

```
# Usage:
progress_cb = ProgressBarCallback(total_timesteps=total_steps,
    ↪print_freq=print_freq)

checkpoint_cb = CheckpointCallback(save_freq=save_freq, save_path=LOG_DIR,
    ↪name_prefix="ppo_mc")
model.learn(total_timesteps=total_steps, callback=[progress_cb, checkpoint_cb])
vec_env.close()
```

Training progress: 10.0% (100 / 1000 steps)

Training progress: 20.0% (200 / 1000 steps)

```
-----
| rollout/          |          |
|   ep_len_mean    | 1.34     |
|   ep_rew_mean    | -44.7     |
| time/            |          |
|   fps            | 0         |
|   iterations      | 1         |
|   time_elapsed    | 739      |
|   total_timesteps | 512      |
-----
```

Training progress: 30.0% (300 / 1000 steps)

Training progress: 40.0% (400 / 1000 steps)

Training progress: 50.0% (500 / 1000 steps)

```
-----
| rollout/          |          |
|   ep_len_mean    | 1.4       |
|   ep_rew_mean    | -44.7     |
| time/            |          |
|   fps            | 0         |
|   iterations      | 2         |
|   time_elapsed    | 1422      |
|   total_timesteps | 1024      |
| train/           |          |
|   approx_kl       | 0.017515296 |
|   clip_fraction   | 0.26      |
|   clip_range      | 0.2       |
|   entropy_loss    | -2.76     |
|   explained_variance | 0.00497   |
|   learning_rate    | 0.0003    |
|   loss            | 703       |
|   n_updates        | 10        |
|   policy_gradient_loss | -0.0494   |
|   value_loss       | 1.76e+03   |
-----
```

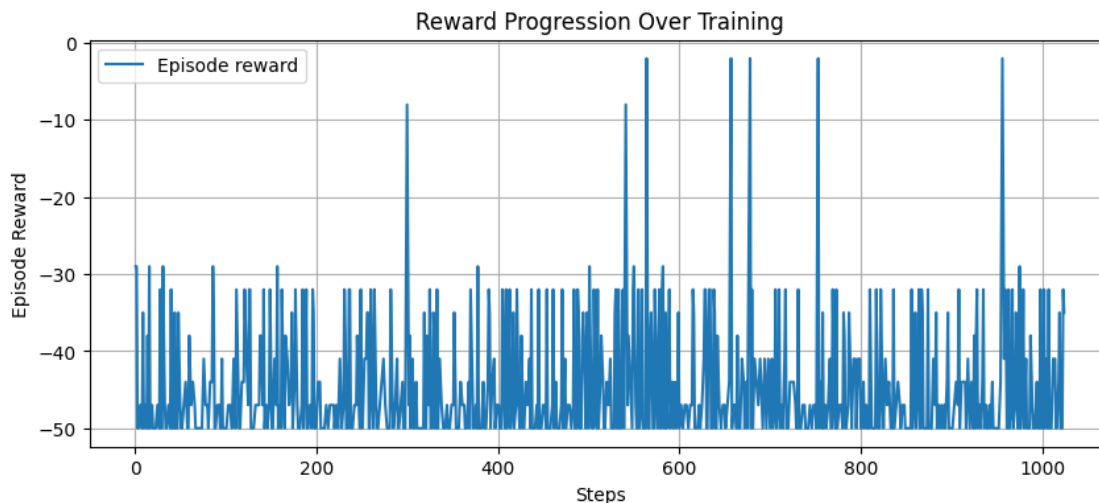
1.10 Visualizing Reward Progression

After training, we can visualize how the agent's episode rewards evolve over time using the monitor logs saved by VecMonitor. The monitor CSV files record episode rewards, lengths, and cumulative steps.

```
[13]: import pandas as pd
import matplotlib.pyplot as plt
import glob

# Find the latest monitor file
monitor_files = glob.glob(f"{LOG_DIR}/monitor.csv")
assert len(monitor_files) > 0, "No monitor file found"
monitor_file = monitor_files[-1] # Use the latest

df = pd.read_csv(monitor_file, skiprows=1)
plt.figure(figsize=(10, 4))
plt.plot(df['l'].cumsum(), df['r'], label="Episode reward") # Steps vs. reward
plt.xlabel('Steps')
plt.ylabel('Episode Reward')
plt.title('Reward Progression Over Training')
plt.legend()
plt.grid()
plt.show()
```



```
[14]: window = 20 # or whatever makes sense
df['rolling_reward'] = df['r'].rolling(window).mean()
plt.figure(figsize=(10, 4))
plt.plot(df['l'].cumsum(), df['rolling_reward'], label=f"Smoothed ({window}-ep) ↵
↵Reward")
```

```
plt.xlabel('Steps')
plt.ylabel('Episode Reward')
plt.title('Reward Progression Over Training (Smoothed)')
plt.legend()
plt.grid()
plt.show()
```

